

Laborator 8

Semafoare

8.1 Scopul lucrării

Laboratorul curent prezintă conceptul de semafor, împreună cu cele mai importante apeluri de sistem Linux pentru manipularea semafoarelor.

8.2 Fundamente teoretice

8.2.1 Conceptul de semafor

Semaforul este un mecanism de sincronizare a execuției mai multor procese ce folosesc aceeași resursă. Zona din codul unui program în care resursa comună este utilizată se numește *regiune critică*. De obicei, accesul unui proces la regiunea critică trebuie restricționat, astfel încât resursa comună să se găsească mereu într-o stare consistentă.

Componentele principale ale unui semafor sunt:

- un întreg **perm**, ce reprezintă numărul de permisiuni asociate cu semaforul respectiv;
- o coadă **q**, ce reprezintă lista de procese ce așteaptă să primească permisiuni de la semaforul respectiv.

Cele două componente nu vor fi manipulate direct, ci prin intermediul următoarelor primitive:

- *wait* (denumită și *down* sau *P*)
Această primitivă încearcă să decrementeze numărul de permisiuni, dacă acest număr este o valoare pozitivă. Altfel, procesul curent este pus în coada de așteptare și suspendat.

```
while (perm == 0) {  
    q.push(crt_proc);  
    crt_proc.suspend();  
}  
perm = perm - 1;
```

- *post* (denumită și *up* sau *V*)
Se incrementează numărul de permisiuni, iar primul proces din coadă este trezit pentru a-și continua execuția.

```
perm = perm + 1;  
if (!q.empty()) {  
    proc = q.pop();  
    proc.resume();  
}
```

Se consideră că pseudocodul pentru cele două operații de mai sus se execută în mod atomic.

8.2.2 Crearea, deschiderea și închiderea semafoarelor

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);

int sem_close(sem_t *sem);
int sem_unlink(const char *name);
```

Pentru a compila programe ce folosesc aceste funcții, `gcc` trebuie să primească în plus opțiunea `-pthread`.

8.2.2.1 sem_open

Deschiderea unui semafor (cu nume) pe Linux se face în mod similar cu deschiderea unui fișier. Dacă mai multe procese apelează `sem_open` pe același nume, vor primi referințe la același semafor.

Parametrul `oflag` reprezintă o mască pe biți ce conține valori privind modul de deschidere:

- `O_CREAT` - în cazul în care semaforul nu există deja, este creat;
- `O_EXCL` - dacă folosim `O_CREAT` și semaforul există deja, apelul de sistem va eșua.

În cazul în care folosim flag-ul `O_CREAT`, va trebui să specificăm încă doi parametri, `mode` și `value`.

Parametrul `mode` reprezintă permisiunile semaforului, care se specifică în mod similar cu cele pentru un fișier la apelul de sistem `open`.

Parametrul `value` reprezintă numărul de permisiuni inițiale ale semaforului.

8.2.2.2 sem_close

Apelul de sistem `sem_close` este utilizat pentru a închide referința spre un semafor. Semaforul va continua să existe în sistem și va putea fi re-deschis ulterior.

8.2.2.3 sem_unlink

Acest apel de sistem va șterge semaforul din sistem. Dacă mai există referințe spre el, semaforul se va șterge doar când ultima referință se închide, dar semaforul nu mai poate fi deschis între timp.

8.2.2.4 Exemplu

Dorim să scriem o funcție care adaugă la finalul unui fișier un mesaj dat (*logging*). Această funcție trebuie să poată fi apelată din programe diferite pentru a scrie în același fișier, fără ca fișierul să devină corupt. În acest caz, resursa comună este fișierul, iar regiunea critică va fi codul ce operează asupra fișierului.

Vom începe prin a deschide și închide un semafor cu nume, care va proteja regiunea critică.

```
1 void write_log(const char *message)
2 {
3     sem_t *logSem = NULL;
4
5     logSem = sem_open("/l08p1_log_semaphore", O_CREAT, 0644, 1);
6     if(logSem == NULL) {
7         perror("Could not acquire the semaphore");
8         return;
9     }
10
11     ...
```

```

12 |
13 |     sem_close(logSem);
14 | }

```

La linia 3 se declară o variabilă de tip `sem_t*`, care va pointa spre structura de tip semafor.

La linia 5 apelăm `sem_open` pentru a obține acces la semafor. Mai multe programe care vor să intre în aceeași regiune critică trebuie să acceseze același semafor. Din acest motiv, numele semafoarelor sunt vizibile la nivelul întregului sistem. Numele semaforului trebuie să înceapă cu `/'` și să nu conțină alt caracter `/'`.

În acest punct, nu putem ști dacă suntem primii care accesează semaforul sau acesta există deja. Vom folosi valoarea `O_CREAT` pentru parametrul `oflag`, care are semantica de a crea semaforul doar dacă nu există deja.

În cazul în care semaforul este creat, se vor lua în considerare și ultimii doi parametri, respectiv permisiunile inițiale și valoarea inițială. Vom seta valoarea inițială pe 1, pentru a permite intrarea în regiunea critică a unui singur proces, la un moment dat.

În cazul în care nu se reușește obținerea accesului la semafor, `sem_open` returnează `NULL`, caz în care `perror` ne poate indica motivul eșecului.

La finalul funcției se va apela `sem_close` (linia 13), pentru a închide semaforul.

8.2.3 Operații pe semafoare

```

#include <semaphore.h>

int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);

int sem_post(sem_t *sem);

```

8.2.3.1 sem_wait

Acest apel de sistem marchează intrarea în regiunea critică și corespunde primitivei *wait* descrisă în secțiunea 8.2.1. La apelul `sem_wait` procesul sau thread-ul curent poate să își continue execuția, obținând permisiunea de la semafor, sau să fie blocat și pus în coada de așteptare până când permisiunea cerută va fi disponibilă.

8.2.3.2 sem_post

Acest apel de sistem marchează ieșirea din regiunea critică și returnează permisiunea obținută prin `sem_wait`. El corespunde primitivei *post* descrisă în secțiunea 8.2.1.

8.2.3.3 sem_trywait

Există situații în care vrem să obținem o permisiune de la un semafor, dar, în cazul în care aceasta nu este disponibilă, să nu fim blocați, ci să putem continua execuția.

Apelul de sistem `sem_trywait` are același efect cu `sem_wait`, dacă permisiunea e disponibilă, altfel nu va bloca procesul sau thread-ul curent, ci va returna o valoare de eroare diferită de 0.

8.2.3.4 Exemplu

Vom continua exemplul din secțiunea 8.2.2.4 pentru sincronizarea scrierii în fișier.

```

1 | void write_log(const char *message)
2 | {
3 |     sem_t *logSem = NULL;
4 |     int fd = -1;
5 |
6 |     logSem = sem_open("/l08p1_log_semaphore", O_CREAT, 0644, 1);
7 |     if(logSem == NULL) {
8 |         perror("Could not acquire the semaphore");

```

```

9      return;
10     }
11
12     //entering the critical region
13     sem_wait(logSem);
14
15     fd = open("log.txt", O_CREAT | O_WRONLY | O_APPEND, 0644);
16     if(fd < 0) {
17         perror("Could not open log for writing");
18     } else {
19         write(fd, message, strlen(message));
20         close(fd);
21     }
22
23     //exiting the critical region
24     sem_post(logSem);
25
26     sem_close(logSem);
27 }

```

Regiunea critică este delimitată de apelurile `sem_wait` de la linia 13 și `sem_post` de la linia 24. Două thread-uri sau procese diferite care folosesc acest semafor nu pot fi simultan cu execuția între `sem_wait` și `sem_post`.

În interiorul regiunii critice se poate scrie cod normal care accesează resursa. Chiar dacă întâlnim situații de eroare, nu este recomandat să părăsim funcția din interiorul regiunii critice (apelând `return`), deoarece nu am mai ajunge să apelăm `sem_post`, iar semaforul ar rămâne blocat.

8.2.4 Semafoare anonime

```

#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);

```

Semafoarele anonime funcționează pe același principiu cu semafoarele cu nume, cu excepția faptului că sunt inițializate în procesul curent și se folosesc pentru a sincroniza thread-urile acestuia (deși se pot transmite și altor procese, dar numai folosind memoria partajată).

8.2.4.1 sem_init

Apelul de sistem primește ca prim parametru un pointer la o variabilă de tip semafor, pe care o va inițializa. Inițializarea unui semafor se face **o singură dată**.

Parametrul `pshared` trebuie să aibă valoarea 0 dacă vrem să folosim semaforul doar în procesul curent sau 1 dacă vrem să poată fi transmis și altor procese.

Ultimul parametru reprezintă valoarea inițială a semaforului, fiind similar cu parametrul cu același nume din `sem_open`.

8.2.4.2 sem_destroy

Acest apel de sistem primește ca argument un pointer la un semafor inițializat cu `sem_init` și eliberează resursele alocate acestuia.

8.2.4.3 Exemplu

Vom prezenta un exemplu similar cu cel de la semafoarele cu nume, dar vom folosi un semafor anonim pentru a sincroniza accesul la fișier pentru mai multe thread-uri din procesul curent. Fiecare thread va scrie câteva mesaje în fișier, așteptând un timp aleator între scrieri.

În primul rând, pentru a putea transmite informații thread-urilor, vom defini structura `TH_STRUCT`, care conține informațiile ce se doresc transmite. În mod particular, vom avea câte o valoare care să apară în mesaj și un pointer la semafor.

```
1 typedef struct {
2     int value;
3     sem_t *logSem;
4 } TH_STRUCT;
```

În funcția `main()` se va inițializa semaforul, se vor crea thread-urile și se va aștepta terminarea acestora.

```
1 int main(void)
2 {
3     sem_t logSem;
4     int i;
5     TH_STRUCT params[NR_THREADS];
6     pthread_t tids[NR_THREADS];
7
8     //initialize random number generator
9     srand(time(NULL));
10    //initialize the semaphore
11    if(sem_init(&logSem, 0, 1) != 0) {
12        perror("Could not init the semaphore");
13        return -1;
14    }
15    //create the threads
16    for(i=0; i<NR_THREADS; i++) {
17        params[i].value = 10 * (i + 1);
18        params[i].logSem = &logSem;
19        pthread_create(&tids[i], NULL, thread_function, &params[i]);
20    }
21    //wait for the threads to finish
22    for(i=0; i<NR_THREADS; i++) {
23        pthread_join(tids[i], NULL);
24    }
25    //destroy the semaphore
26    sem_destroy(&logSem);
27
28    return 0;
29 }
```

La linia 9 se inițializează generatorul de numere aleatoare, pe baza timpului curent.

La linia 11 se inițializează semaforul `logSem`. Observați că acesta a fost declarat la linia 3 ca `sem_t`, nu ca `sem_t*`, ca în cazul semafoarelor cu nume.

Liniile 16–20 conțin bucla de creare a thread-urilor. În primul rând se inițializează structura cu parametrii fiecărui thread (liniile 17–18), apoi se apelează `pthread_create` ce completează id-ul fiecărui thread creat în vectorul `tids`.

Următoarea buclă (liniile 22–24) așteaptă terminarea fiecărui thread. Deoarece nu ne interesează valorile returnate de funcțiile de thread, putem seta parametrul al doilea pe `NULL`.

```
1 void *thread_function(void *p)
2 {
3     TH_STRUCT *s = (TH_STRUCT*)p;
4     int i, count;
5     char message[20];
6
7     for(i=0; i<3; i++) {
8         usleep(1000 * (rand() % 20));
```

```

9      count = s->value + i + 1;
10     snprintf(message, 20, "Log entry %d\n", count);
11     write_log(message, s->logSem);
12 }
13 return NULL;
14 }
```

Funcția `thread`-urilor definită mai sus începe prin a face cast la pointer-ul primit către un pointer la `TH_STRUCT`, pentru a-și putea accesa parametrii.

În bucla de la liniile 7–12 se apelează scrierea de mesaje în fișier, la intervale aleatorii.

Pentru a aștepta un timp aleatoriu folosim funcția `usleep` care așteaptă numărul de microsecunde primit ca parametru. Valoarea dată va reprezenta un timp aleator între 0 și 19 milisecunde.

Liniile 9–10 construiesc mesajul ce trebuie afișat, folosind funcția `snprintf`, apoi acesta se scrie în log, la linia 11.

```

1 void write_log(const char *message, sem_t *logSem)
2 {
3     int fd = -1;
4     //entering the critical region
5     sem_wait(logSem);
6     fd = open("log2.txt", O_CREAT | O_WRONLY | O_APPEND, 0644);
7     if(fd < 0) {
8         perror("Could not open log for writing");
9     } else {
10        write(fd, message, strlen(message));
11        close(fd);
12    }
13    //exiting the critical region
14    sem_post(logSem);
15 }
```

Funcția `write_log` este similară cu cea din secțiunile anterioare, diferența fiind că nu se mai deschide semaforul, acesta fiind primit ca parametru.

8.3 Mersul lucrării

8.3.1 Întrebări recapitulative

1. Ce se întâmplă când apelăm `sem_wait` pe un semafor cu 0 permisiuni?
2. În ce condiții alegem să folosim un semafor cu nume față de unul anonim?
3. Care este diferența dintre `sem_close` și `sem_unlink`?
4. Pentru a sincroniza mai multe thread-uri se folosește un semafor comun pe care se apelează `sem_wait` la începutul funcției și `sem_post` la final. De ce este în general greșită această abordare?
5. Codul de mai jos se apelează pe mai multe thread-uri pentru a construi un vector global `v` cu numere prime. Explicați ce se întâmplă dacă un thread este preemptat între liniile 3 și 4 și se execută codul unui alt thread? Cum ați folosi un semafor pentru a evita cursa critică?

```

1 for(i=start; i<=end; i++) {
2     if(is_prime(i)) {
3         size++;
4         v[size] = i;
5     }
6 }
```

8.3.2 Probleme rezolvate

1. (*l08p1.log.c*) Să se scrie un program care scrie mai multe mesaje adăugând la finalul unui fișier, accesul la fișier fiind sincronizat printr-un semafor cu nume. Programul este descris în secțiunile 8.2.2.4 și 8.2.3.4.
2. (*l08p2.digits.c*) Să se scrie un program care scrie mesaje într-un fișier pe mai multe thread-uri, sincronizând accesul la fișier printr-un semafor anonim. Observați ordinea mesajelor scrise la execuția programului. Programul este descris în secțiunea 8.2.4.3.

8.3.3 Probleme propuse

1. (25p) Scrieți un program care conține o variabilă globală `long count`, ce are valoarea inițială 0. Creați N thread-uri, fiecare executând funcția de mai jos:

```

1 void *thread_function(void *unused)
2 {
3     int i;
4     long aux;
5
6     for(i=0; i<M; i++){
7         aux = count;
8         aux++;
9         usleep(random() % 10);
10        count = aux;
11    }
12    return NULL;
13 }
```

Se pot alege valorile $N = 4$ și $M = 1000$.

- (a) Ce valoare ar trebui să aibă variabila `count` la final?
 - (b) Afișați valoarea variabilei `count` la finalul programului. De ce diferă de valoarea așteptată?
 - (c) Sincronizați thread-urile astfel încât variabila `count` să aibă la final valoarea așteptată.
2. (25p) Scrieți un program care generează 16 thread-uri care execută în mod concurrent funcția de mai jos:

```

1 void *limited_area(void *unused)
2 {
3     nrThreads++;
4     usleep(100);
5     printf("The number of threads in the limited area is: %d\n", nrThreads);
6     nrThreads--;
7     return NULL;
8 }
```

`nrThreads` este o variabilă globală de tip `int`. Sincronizați thread-urile, astfel încât numărul maxim de thread-uri afișat în zona limitată să fie n , unde n se primește ca argument în linia de comandă. Având în vedere soluția de la problema precedentă, sincronizați corect accesul la variabila `nrThreads`!

3. (25p) Scrieți un program care generează N thread-uri sau N procese, fiecare din acestea afișând într-o buclă infinită propriul id, apoi așteptând un timp aleatoriu. Sincronizați cele N thread-uri sau procese, astfel id-urile să se afișeze alternativ (de exemplu dacă avem 3 procese se va afișa 1 2 3 1 2 3 1 2 ...).
Indicație: Pentru a vă asigura că textul afișat cu `printf` ajunge pe ecran la momentul respectiv, după apelul de `printf` apăsați `fflush(stdout)`.

4. (25p) Simulați utilizând thread-uri modul în care atomii de Na și Cl interacționează pentru a forma molecula de sare. Fiecare thread modelează un atom care se creează în mod aleator (fie de tip Na , fie de tip Cl). Atomul respectiv trebuie să aștepte apariția unui atom de tipul opus, după care ambii formează molecula și dispar (afișează cu care thread s-a format molecula, apoi se încheie thread-ul curent). Atomii de același tip trebuie să fie “consumați” în ordinea în care au ajuns la punctul de formare al moleculei. Pentru a valida implementarea, verificați faptul că dacă un atom de Na cu id-ul x afișează că a format molecula cu un atom de Cl cu id-ul y , atunci și atomul de Cl cu id-ul y trebuie să afișeze că a format molecula cu atomul de Na cu id-ul x . Cele două mesaje de la cele două thread-uri pentru atomi ar trebui să fie consecutive.

Bonus: (7p) Aceeași cerință, pentru molecula H_2O .

Bonus 2: (8p) Aceeași cerință, pentru molecula H_2SO_4 .